EvoRec: Self-Evolving Agentic Recommender Systems

Lingyu Mu*
Alibaba International Digital Commerce Group
Beijing, China
moulingyu.mly@alibaba-inc.com

Hao Deng*
Alibaba International Digital Commerce Group
Beijing, China
denghao.deng@alibaba-inc.com

Haibo Xing
Alibaba International Digital Commerce Group
Hangzhou, China
xinghaibo.xhb@alibaba-inc.com

Jinxin Hu†
Alibaba International Digital Commerce Group
Beijing, China
jinxin.hjx@alibaba-inc.com

Yu Zhang
Alibaba International Digital Commerce Group
Beijing, China
daoji@alibaba-inc.com

Xiaoyi Zeng
Alibaba International Digital Commerce Group
Hangzhou, China
yuanhan@taobao.com

Abstract

Optimizing modern recommender systems still relies heavily on engineers iterating by hand, which is slow and bounded by individual expertise. LLM-based agents open a path toward automating this loop, yet existing approaches use the agent only as a code translator that accumulates no methodology, and confine the search to a predefined space that rarely introduces structurally new ideas. We propose EvoRec, a multi-agent framework that co-evolves the recommendation model and the optimization methodology driving it. Four collaborating agents carry out a dual-track loop: the Research Agent and Code Agent iterate the model each round, while the Skill Evolver periodically distills reusable methodology from a persistent Memory of past experiments. Experiments on a public benchmark and one large-scale industrial dataset show that EvoRec improves offline metrics by up to 5.54% over the strongest baseline. An online A/B test further delivers a 1.85% revenue lift and a 1.02% CTR gain, demonstrating the potential to replace traditional manual optimization workflows.

CCS Concepts

• **Information systems** → **Retrieval models and ranking**.

Keywords

Recommender Systems, Agent, Self-evolving

ACM Reference Format:
Lingyu Mu, Hao Deng, Haibo Xing, Jinxin Hu, Yu Zhang, and Xiaoyi Zeng. 2026. EvoRec: Self-Evolving Agentic Recommender Systems. In . ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

*Contributed equally to this research.
†Corresponding authors.

Conference'17, Washington, DC, USA

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Recommender systems serve as the backbone connecting users with information on Internet platforms [13, 22, 24]. Modern recommender systems have evolved from collaborative filtering into sophisticated multi-stage pipelines [12, 27, 30], where each stage is shaped by extensive feature engineering, architecture design, and hyperparameter tuning [10, 14, 26, 32, 35]. This process is inefficient and bounded by individual expertise, leaving large portions of the optimization space underexplored [4, 8].

Recent advances in large language models (LLMs) have enabled agents with multi-step planning, tool invocation, and code generation capabilities to autonomously carry out complex optimization workflows much like human engineers [9, 21, 31]. However, existing efforts still suffer from three limitations: **(1) Shallow agent involvement.** Most methods use the agent only for code generation while humans specify the optimization direction, leaving the agent's reasoning and planning capacities underutilized [5, 16]. **(2) Static experience components.** Some works introduce Skill or Memory modules [3, 17, 29], but these remain frozen after deployment, causing the same failures to recur and preventing successful strategies from being reused. **(3) Confined evolution space.** A few works [2, 23] explore self-evolution [11, 15, 34], but are restricted to hyperparameter search within a predefined space, unable to introduce novel methods.

Based on these observations, we aim to build a self-evolving recommender system that satisfies three properties: **(1) Open-domain exploration**: capable of introducing structurally new methods and insights. **(2) Human-free operation**: the entire pipeline is autonomously driven by collaborating agents. **(3) Self-improving experience**: the system continuously distills knowledge from its own evolution trajectory, growing stronger over time.

To address these challenges, we propose EvoRec, a self-evolving framework for recommender systems. EvoRec comprises three core components: a self-evolving Model, a self-evolving Skill library, and a persistent Memory store. Memory persists the complete trajectory of each iteration in structured form, including the optimization hypothesis, code modifications, training logs, evaluation metrics, and success/failure attribution. At the execution level, EvoRec employs four collaborating agents. The Orchestrator agent coordinates the overall optimization loop and drives two self-evolution pathways:

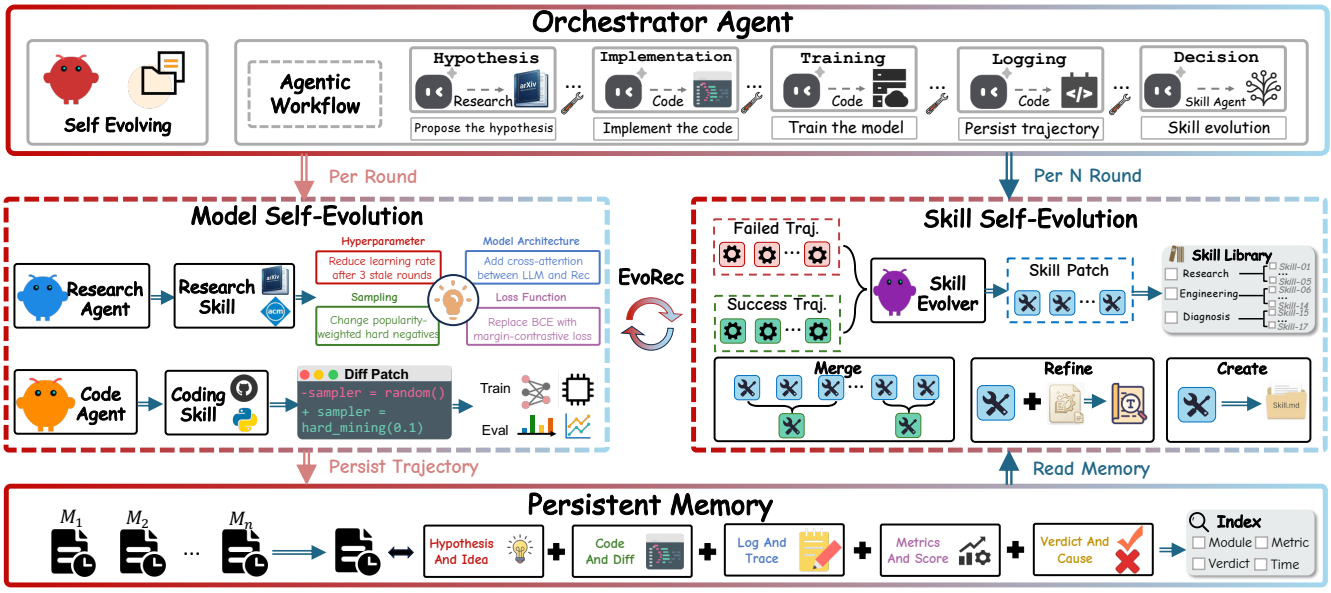

Figure 1: The overview of EvoRec. Four collaborating agents drive dual-track self-evolution: the Research Agent and Code Agent perform per-round model self-evolution, while the Skill Evolver periodically distills methodologies from persistent Memory to update the Skill library.

- (1) Model self-evolution is driven by the Research Agent and Code Agent. The Research Agent retrieves external knowledge (e.g., arXiv) and combines it with Skill and Memory to generate optimization hypotheses. The Code Agent translates each hypothesis into code edits and submits training.
- (2) Skill self-evolution is handled by the Skill Evolver, which distills reusable methodologies from Memory every N rounds and updates the Skill library.

The model iterates through a hypothesis–implementation–validation cycle each round, while Skills evolve periodically after sufficient experience has accumulated. Under the coordination of the Orchestrator, the two pathways are coupled through persistent Memory, forming a self-reinforcing loop in which the model grows stronger and the methodologies become increasingly refined. Experiments on a public benchmark and one industrial dataset show up to 5.54% offline improvement over the strongest baselines, and an online A/B test confirms a 1.85% revenue lift and 1.02% CTR gain.

Our contributions can be summarized as follows:

- We propose EvoRec, the first multi-agent self-evolving optimization framework for recommender systems, enabling continuous model self-evolution without human intervention.
- We design a dual-track self-evolution mechanism driven by four collaborating agents, achieving co-evolution of Model and Skill through persistent Memory.
- We validate the effectiveness of EvoRec on a public benchmark and an industrial advertising platform, and further demonstrate its practical deployment value through online A/B testing.

2 Method

2.1 Overview

As illustrated in Figure 1, EvoRec consists of three core components: the Model, an evolvable Skill library, and a structured Memory, coordinated by four collaborating agents. Each iteration sequentially completes hypothesis generation, code implementation, training, and experience persistence, forming a dual-track loop where the model and methodologies evolve in tandem.

2.2 Orchestrator Agent

The Orchestrator maintains global state and coordinates the other three agents. At each iteration, it activates the Research Agent and Code Agent to complete one round of model optimization, then writes the full trajectory into Memory. Every N rounds, it triggers the Skill Evolver to update the Skill library. The loop terminates when reaching $T_{\max}$ iterations, k consecutive rounds without improvement, or a target metric threshold. Failed rounds are preserved in Memory as negative examples rather than discarded.

2.3 Model Self-Evolution

Model self-evolution is jointly carried out by the Research Agent and the Code Agent, forming the core of each EvoRec iteration. Let the global state at round t be $\langle \theta_{t-1}, \mathcal{S}_t, \mathcal{M}_t \rangle$, denoting the current model parameters, the Skill library, and the Memory experience store, respectively.

2.3.1 Research Agent. The Research Agent is responsible for generating the optimization hypothesis for the current round. It retrieves candidate research directions from external knowledge sources $\mathcal{K}_{\text{ext}}$ (e.g., arXiv, technical blogs), weighs them against the Research-type

methodologies in $\mathcal{S}_t$ and the historical success and failure cases in $\mathcal{M}_t$, and outputs:

$$h_t \sim \pi_R(\cdot \mid \mathcal{K}_{\text{ext}}, \mathcal{S}_t, \mathcal{M}_t), \tag{1}$$

where $h_t = \langle motive, target\_module, expected\_gain \rangle$ comprises the optimization motive, the target module to modify, and the expected gain. Before generating a hypothesis, the Research Agent performs a diversity constraint check to avoid focusing on the same module for too many consecutive rounds, which could lead to local optima. Retrieval over $\mathcal{M}_t$ allows it to steer clear of previously failed directions, while injection of $\mathcal{S}_t$ biases it toward exploration patterns that have proven effective.

2.3.2 Code Agent. The Code Agent translates the hypothesis h_t into an executable experiment. It locates the relevant code, invokes Engineering-type methodologies in $\mathcal{S}_t$ to generate minimal-diff modifications, and submits the training job. Upon completion, it outputs the round result r_t containing training logs and evaluation metrics. If h_t is deemed infeasible during implementation, a rejection signal is returned and the Research Agent generates an alternative hypothesis. Failed runs are marked with failure reasons and preserved for subsequent Skill evolution.

2.4 Memory: Persistent Experience Storage

Memory is the experience hub of EvoRec. It persists the complete trajectory of each iteration in structured form, serving both as a retrieval source for the Research Agent and as distillation material for the Skill Evolver. After each iteration, the Orchestrator packages the round's output into a memory entry:

$$m_t = \langle h_t, \Delta c_t, log_t, metric_t, verdict_t \rangle, \tag{2}$$

where h_t is the optimization hypothesis, Δc_t the code modifications, log_t the training log summary, $metric_t$ the offline evaluation metric vector, and $verdict_t \in \{\text{success}, \text{fail}, \text{neutral}\}$ the attribution label. All entries are appended chronologically to form $\mathcal{M}_t = \{m_1, m_2, \ldots, m_t\}$.

To support efficient access by upstream agents, Memory builds a structured index upon each write. Specifically, metadata fields are extracted from each m_t, including *target_module* (the modified model module), *verdict* (the attribution label), *metric_delta* (the metric change relative to the previous version), and *timestamp*. These fields are stored as an inverted index that supports exact field-level filtering. Memory itself does not participate in self-evolution and serves purely as passive storage.

2.5 Skill Self-Evolution

Unlike the per-round experiment trajectories stored in Memory, Skills are robust operational principles produced by cross-round inductive compression rather than local experience. They are expressed in declarative natural language, not tied to any specific model architecture, and can be reused across different optimization objectives. Formally, a Skill is defined as a four-tuple $s = \langle name, scope, content, evidence \rangle$, where *name* is the skill identifier, *scope* specifies the applicable conditions (e.g., "attention selection" or "loss not decreasing"), *content* provides the concrete operational guidelines whose granularity ranges from a single rule of thumb to a multi-step conditional procedure and grows richer as evolution progresses, and *evidence* points back to the Memory entries that support the rule to ensure traceability. A short example is:

> **Skill #12**: Hard Negative Mining
> *Scope*: Sampling strategy when recall plateaus
> *Content*: When R@10 stagnates for ≥3 rounds, replace random negatives with popularity-weighted hard negatives ($\tau \in [0.05, 0.15]$). Start with τ=0.1 and adjust based on training loss variance.
> *Evidence*: Memory entries m_{14}, m_{18}, m_{22}

By functional stage, Skills are divided into three categories: Research Skills (guiding hypothesis formulation), Engineering Skills (guiding code implementation), and Diagnosis Skills (guiding failure repair, distilled primarily from *verdict*=fail cases). The Orchestrator matches each Skill's *scope* against the current objective and injects only relevant entries into the agent's context. All categories share the same evolution pipeline, collectively forming $\mathcal{S}_t$.

2.5.1 Triggering and Batch Selection. Skill self-evolution is carried out by the Skill Evolver agent and triggered by the Orchestrator every N rounds of Model iteration. At the triggering round t, the Skill Evolver selects the most recent N rounds of experiment records from Memory as the distillation batch:

$$\mathcal{B}_t = \{m_\tau \in \mathcal{M}_t \mid t - N + 1 \le \tau \le t\}, \tag{3}$$

and applies balanced filtering by *verdict* label so that success, fail, and neutral cases appear in the batch at a preset ratio ρ. This balance ensures that the distillation process extracts positive patterns from successful cases while also consolidating lessons from failures.

2.5.2 Distillation and Update. Given batch $\mathcal{B}_t$ and the current Skill library $\mathcal{S}_t$, the Skill Evolver generates a set of candidate Skill patches via policy π_E:

$$C_t = \{c_1, \ldots, c_K\} \sim \pi_E(\cdot \mid \mathcal{B}_t, \mathcal{S}_t). \tag{4}$$

Each candidate c_k follows the same four-tuple structure. For each candidate, the system computes its maximum similarity to the existing Skill library $\sigma_k = \max_{s \in \mathcal{S}_t} \text{sim}(c_k, s)$, and applies one of three update operators based on threshold τ_c:

- **Merge**: Candidates that are pairwise similar above τ_c within the batch are first consolidated into a single candidate, preventing redundant Skills from being introduced simultaneously.
- **Refine**: When $\sigma_k \ge \tau_c$, c_k is merged with its most similar existing Skill and revised, corresponding to methodology refinement.
- **Create**: When $\sigma_k < \tau_c$, candidate c_k is added to the library as a new Skill, corresponding to methodology expansion.

The Skill library is then updated as:

$$\mathcal{S}_{t+1} = (\mathcal{S}_t \setminus \mathcal{S}_{\text{refined}}) \cup \mathcal{S}'_{\text{refined}} \cup \mathcal{S}_{\text{new}}. \tag{5}$$

3 Experiment

3.1 Experimental Setup

3.1.1 Datasets and Metrics. We evaluate on one public dataset (Amazon Books [6], filtered to users with ≥5 interactions) and one industrial dataset from the internal interaction logs of a Southeast Asian e-commerce platform spanning January to May 2026, containing approximately 2.95B user-item interactions, 14.7M users,

Table 1: Performance comparison on the Books and Industrial datasets. "Imp." shows the relative improvement (%) over the strongest baseline in each group. Best results per group are in bold and second-best are underlined.

		ID-based Recommendation							*Generative Recommendation*						
		SASRec	S^3Rec	PFormer	MGUI	Auto-MGUI	Evo-MGUI	Imp.	HSTU	TIGER	Cobra	REG4Rec	Auto-REG	Evo-REG4Rec	Imp.
Books	R@5	0.0472	0.0471	0.0492	0.0503	0.0506	**0.0527**	**4.15%**	0.0536	0.0562	0.0584	0.0587	0.0592	**0.0608**	**2.70%**
	N@5	0.0267	0.0264	0.0281	0.0284	0.0279	**0.0298**	**4.93%**	0.0307	0.0324	0.0335	0.0331	0.0327	**0.0349**	**4.18%**
	R@10	0.0599	0.0597	0.0623	0.0635	0.0630	**0.0664**	**4.57%**	0.0697	0.0733	0.0761	0.0768	0.0763	**0.0792**	**3.13%**
	N@10	0.0343	0.0339	0.0357	0.0361	0.0358	**0.0381**	**5.54%**	0.0392	0.0415	0.0429	0.0425	0.0431	**0.0446**	**3.48%**
Industry	R@5	0.0701	0.0706	0.0734	0.0741	0.0745	**0.0775**	**4.03%**	0.0832	0.0904	0.0965	0.1016	0.1008	**0.1053**	**3.64%**
	N@5	0.0919	0.0939	0.0984	0.0982	0.0976	**0.1037**	**5.39%**	0.1062	0.1165	0.1236	0.1294	0.1302	**0.1331**	**2.23%**
	R@10	0.0518	0.0523	0.0544	0.0547	0.0542	**0.0571**	**4.39%**	0.0613	0.0672	0.0701	0.0736	0.0731	**0.0761**	**3.40%**
	N@10	0.0681	0.0697	0.0741	0.0736	0.0743	**0.0776**	**4.44%**	0.0811	0.0871	0.0916	0.0957	0.0961	**0.0993**	**3.33%**

and 20.1M items. We adopt leave-one-out evaluation and report Recall@5/10 (R@5/10) and NDCG@5/10 (N@5/10) [27].

3.1.2 Baselines and Base Models. We compare against *ID-based recommenders* (SASRec [8], S^3Rec [36], PinnerFormer [18]), *generative recommenders* (HSTU [33], TIGER [20], Cobra [30]), and *auto-optimization* (AutoML [7], applied to each base model respectively as Auto-MGUI and Auto-REG). We select MGUI [25] (ID-based) and REG4Rec [28] (generative) as base models for EvoRec, yielding Evo-MGUI and Evo-REG4Rec.

3.1.3 Implementation Details. All models are trained with PyTorch on 4 NVIDIA A100 GPUs [19]. The four agents share Claude Opus 4.6 as the reasoning backbone, with N=5, $T_{\max}$=50, k=10, τ_c=0.6, and ρ=(0.5, 0.3, 0.2). Training hyperparameters for both base models follow their original papers. Reproducing the pipeline with Qwen3.7-MAX [1] yields consistent conclusions, confirming robustness to the LLM backbone choice.

3.2 Main Results

Table 1 reports results on the public and industrial datasets. EvoRec-MGUI and EvoRec-REG4Rec achieve the best performance across all datasets and metrics, with up to 5.54% relative improvement over the strongest baseline. The consistent gains across both ID-based and generative paradigms confirm the generality of the framework. The improvement on the industrial dataset is generally larger than on Books, as the richer feature space provides a wider optimization landscape. Meanwhile, gains in the generative paradigm are slightly smaller, as generative models already incorporate stronger representational priors. Compared with AutoML, which is bounded by a fixed configuration space, EvoRec achieves consistent advantages because its Skill self-evolution mechanism accumulates and refines optimization methodologies across iterations, continuously identifying better directions beyond predefined search ranges.

3.3 Ablation Study

To verify the contribution of each core component, we conduct three ablation experiments on the industrial dataset using EvoRec-MGUI as the base, with results reported in Table 2.

- **w/o Skill Evolution** (−2.10% R@10): The Skill library stays frozen. The drop confirms that continuously refined methodologies are critical for guiding later-stage optimization.

Table 2: Ablation study on the Industrial dataset using EvoRec-MGUI. Δ denotes the relative drop in R@10 compared to the full model.

Method	R@5	N@5	R@10	N@10	ΔR@10
EvoRec-MGUI	**0.0775**	**0.1037**	**0.0571**	**0.0776**	–
w/o Skill Evo.	0.0759	0.1015	0.0559	0.0761	-2.10%
w/o Memory	0.0749	0.1003	0.0551	0.0750	-3.50%
w/o Ext. Know.	0.0763	0.1021	0.0562	0.0764	-1.58%

Table 3: Rounds with the most significant R@10 changes during EvoRec-MGUI optimization on the Industrial dataset.

Rnd	Category	Modification	ΔR@10	Source
3	Hyperparam	Reduce lr to 5e-4	+0.41%	Research
11	Loss	Focal loss variant	-0.82%	Research
14	Sampling	Neg. samples 4→16	+0.52%	Research
19	Architecture	Multi-interest pooling (4 heads)	+1.03%	Research
24	Loss	Replace BCE with InfoNCE	+1.18%	Research
27	Sampling	Hard neg. mining (τ=0.1)	**+2.31%**	Skill #12
33	Hyperparam	Layer-wise lr decay	+0.63%	Skill #16
36	Architecture	Attention head pruning	-0.87%	Research

- **w/o Memory** (−3.50% R@10): Without historical records, the system repeatedly explores failed directions, confirming Memory's role in accelerating convergence.
- **w/o External Knowledge** (−1.58% R@10): Relying solely on internal experience limits the system to known patterns, showing that external knowledge introduces structurally novel solutions.

3.4 Case Study

We use the optimization trajectory of EvoRec-MGUI on the industrial dataset to illustrate the typical working behavior of EvoRec across 50 iterations. Table 3 lists the rounds with the most significant metric changes, including both successful and failed attempts.

The trajectory spans diverse categories including hyperparameters, sampling, architecture, and loss design. Not all attempts succeed: rounds 11 and 36 cause regressions but are preserved in Memory as negative examples, helping the Research Agent avoid similar directions later. After sufficient experience accumulates, Skill-driven rounds emerge (27, 33) and both succeed, with round

27 achieving the largest single-round gain (+2.31%) via Skill #12 distilled from prior sampling experiments. This suggests that distilled methodologies provide more reliable directions than open-ended exploration, while the two sources play complementary roles.

3.5 Online A/B Test

We deploy EvoRec-MGUI on the advertising recommendation platform of a leading e-commerce company in Southeast Asia for a 7-day online A/B test. The control group uses the production model MGUI, and the experimental group replaces it with the EvoRec-optimized variant. Each group contains 20% of users sampled uniformly at random. EvoRec-MGUI delivers a **1.85%** lift in advertising revenue and a **1.02%** improvement in CTR, both statistically significant under a two-sided test ($p < 0.05$). These results confirm that the offline gains of EvoRec translate into real business value.

4 Conclusion

In this work, we propose EvoRec, a multi-agent self-evolving framework for recommender systems that replaces manual optimization with autonomous agent collaboration. We identify three limitations of prior agent-based approaches: shallow involvement limited to code translation, static experience that never updates, and evolution confined to predefined search spaces. EvoRec addresses these via a dual-track mechanism where the Model and the Skill library co-evolve through persistent Memory. Extensive offline and online experiments validate the effectiveness of EvoRec across both public and industrial settings. Future work includes multi-stage cascade optimization and cross-task Skill transfer across different recommendation scenarios.

References

[1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609* (2023).

[2] Yin Cheng, Liao Zhou, Xiyu Liang, Dihao Luo, Tewei Lee, Kailun Zheng, Weiwei Zhang, Mingchen Cai, Jian Dong, and Andy Zhang. 2026. Let the Agent Steer: Closed-Loop Ranking Optimization via Influence Exchange. *arXiv preprint arXiv:2603.27765* (2026).

[3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413* (2025).

[4] Hao Deng, Haibo Xing, Kanefumi Matsuyama, Moyu Zhang, Jinxin Hu, Hong Wen, Yu Zhang, Xiaoyi Zeng, and Jing Zhang. 2025. CSMF: Cascaded Selective Mask Fine-Tuning for Multi-Objective Embedding-Based Retrieval. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2122–2131.

[5] Xuegang Hao, Ming Zhang, Alex Li, Xiangyu Qian, Zhi Ma, Yanlong Zang, Shijie Yang, Zhongxuan Han, Xiaolong Ma, Jinguang Liu, et al. 2025. OxygenREC: An Instruction-Following Generative Framework for E-commerce Recommendation. *arXiv preprint arXiv:2512.22386* (2025).

[6] Ruining He and Julian McAuley. 2016. Ups and Downs: Modeling the Visual Evolution of Fashion Trends with One-Class Collaborative Filtering. In *Proceedings of the 25th International Conference on World Wide Web* (Montréal, Québec, Canada) *(WWW '16)*. International World Wide Web Conferences Steering Committee, Republic and Canton of Geneva, CHE, 507–517. doi:10.1145/2872427.2883037

[7] Xin He, Kaiyong Zhao, and Xiaowen Chu. 2021. AutoML: A survey of the state-of-the-art. *Knowledge-based systems* 212 (2021), 106622.

[8] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 IEEE international conference on data mining (ICDM)*. IEEE, 197–206.

[9] Fei Liu, Xinyu Lin, Hanchao Yu, Mingyuan Wu, Jianyu Wang, Qiang Zhang, Zhuokai Zhao, Yinglong Xia, Yao Zhang, Weiwei Li, et al. 2025. Recoworld: Building simulated environments for agentic recommender systems. *arXiv preprint arXiv:2509.10397* (2025).

[10] Qijiong Liu, Jieming Zhu, Quanyu Dai, and Xiao-Ming Wu. 2022. Boosting deep CTR prediction with a plug-and-play pre-trainer for news recommendation. In *Proceedings of the 29th International Conference on Computational Linguistics*. 2823–2833.

[11] Ziyu Ma, Shidong Yang, Yuxiang Ji, Xucong Wang, Yong Wang, Yiming Hu, Tongwen Huang, and Xiangxiang Chu. 2026. Skillclaw: Let skills evolve collectively with agentic evolver. *arXiv preprint arXiv:2604.08377* (2026).

[12] Lingyu Mu, Hao Deng, Haibo Xing, Jinxin Hu, Yu Zhang, Xiaoyi Zeng, and Jing Zhang. 2026Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems. Masked Diffusion Generative Recommendation. *arXiv preprint arXiv:2601.19501* (2026Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems).

[13] Lingyu Mu, Zhengxiao Liu, Zhitong Zhu, and Zheng Lin. 2025. Trust-GRS: A Trustworthy Training Framework for Graph Neural Network Based Recommender Systems Against Shilling Attacks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 12408–12416.

[14] Aashiq Muhamed, Iman Keivanloo, Sujan Perera, James Mracek, Yi Xu, Qingjun Cui, Santosh Rajagopalan, Belinda Zeng, and Trishul Chilimbi. 2021. CTR-BERT: Cost-effective knowledge distillation for billion-parameter teacher models. In *NeurIPS Efficient Natural Language and Speech Processing Workshop*.

[15] Jingwei Ni, Yihao Liu, Xinpeng Liu, Yutao Sun, Mengyu Zhou, Pengyu Cheng, Dexin Wang, Erchao Zhao, Xiaoxi Jiang, and Guanjun Jiang. 2026. Trace2skill: Distill trajectory-local lessons into transferable agent skills. *arXiv preprint arXiv:2603.25158* (2026).

[16] Kesha Ou, Chenghao Wu, Xiaolei Wang, Bowen Zheng, Wayne Xin Zhao, Weitao Li, Long Zhang, Sheng Chen, and Ji-Rong Wen. 2026. Deep Research for Recommender Systems. *arXiv preprint arXiv:2603.07605* (2026).

[17] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. 2023. MemGPT: towards LLMs as operating systems. (2023).

[18] Nikil Pancha, Andrew Zhai, Jure Leskovec, and Charles Rosenberg. 2022. Pinnerformer: Sequence modeling for user representation at pinterest. In *Proceedings of the 28th ACM SIGKDD conference on knowledge discovery and data mining*. 3702–3712.

[19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).

[20] Shashank Rajput, Nikhil Mehta, Anima Singh, Raghunandan Hulikal Keshavan, Trung Vu, Lukasz Heldt, Lichan Hong, Yi Tay, Vinh Tran, Jonah Samost, et al. 2023. Recommender systems with generative retrieval. *Advances in Neural Information Processing Systems* 36 (2023), 10299–10315.

[21] Jiakai Tang, Yujie Luo, Xunke Xi, Fei Sun, Xueyang Feng, Sunhao Dai, Chao Yi, Dian Chen, Zhujin Gao, Yang Li, et al. 2025. Interactive Recommendation Agent with Active User Commands. *arXiv preprint arXiv:2509.21317* (2025).

[22] Hanbing Wang, Xiaorui Liu, Wenqi Fan, Xiangyu Zhao, Venkataramana Kini, Devendra Yadav, Fei Wang, Zhen Wen, Jiliang Tang, and Hui Liu. 2024. Rethinking large language model architectures for sequential recommendations. *arXiv preprint arXiv:2402.09543* (2024).

[23] Haochen Wang, Yi Wu, Daryl Chang, Li Wei, and Lukasz Heldt. 2026. Self-evolving recommendation system: End-to-end autonomous model optimization with LLM agents. *arXiv preprint arXiv:2602.10226* (2026).

[24] Shoujin Wang, Longbing Cao, Yan Wang, Quan Z Sheng, Mehmet A Orgun, and Defu Lian. 2021. A survey on session-based recommender systems. *ACM Computing Surveys (CSUR)* 54, 7 (2021), 1–38.

[25] Bin Wu, Xiaowen Yin, Xun Su, and Mingliang Xu. 2026. Modeling Multi-Grained User Interests for Sequential Recommendation. *IEEE Transactions on Computational Social Systems* (2026).

[26] Chuhan Wu, Fangzhao Wu, Tao Qi, and Yongfeng Huang. 2021. Empowering news recommendation with pre-trained language models. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval*. 1652–1656.

[27] Haibo Xing, Hao Deng, Yucheng Mao, Jinxin Hu, Yi Xu, Hao Zhang, Jiahao Wang, Shizhun Wang, Yu Zhang, Xiaoyi Zeng, et al. 2025. Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems. *arXiv preprint arXiv:2508.15308* (2025).

[28] Haibo Xing, Hao Deng, Yucheng Mao, Jinxin Hu, Yi Xu, Hao Zhang, Jiahao Wang, Shizhun Wang, Yu Zhang, Xiaoyi Zeng, et al. 2025. Reg4rec: Reasoning-enhanced generative model for large-scale recommendation systems. *arXiv preprint arXiv:2508.15308* (2025).

[29] Renjun Xu and Yang Yan. 2026. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430* (2026).

[30] Yuhao Yang, Zhi Ji, Zhaopeng Li, Yi Li, Zhonglin Mo, Yue Ding, Kai Chen, Zijian Zhang, Jie Li, Shuanglong Li, et al. 2025. Sparse meets dense: Unified generative recommendations with cascaded sparse-dense representations. *arXiv preprint arXiv:2503.02453* (2025).

[31] Chao Yi, Dian Chen, Gaoyang Guo, Jiakai Tang, Jian Wu, Jing Yu, Mao Zhang, Wen Chen, Wenjun Yang, Yujie Luo, et al. 2025. RecGPT-V2 Technical Report. *arXiv preprint arXiv:2512.14503* (2025).

[32] Zheng Yuan, Fajie Yuan, Yu Song, Youhua Li, Junchen Fu, Fei Yang, Yunzhu Pan, and Yongxin Ni. 2023. Where to go next for recommender systems? id- vs. modality-based recommender models revisited. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2639–2649.

[33] Jiaqi Zhai, Lucy Liao, Xing Liu, Yueming Wang, Rui Li, Xuan Cao, Leon Gao, Zhaojie Gong, Fangda Gu, Michael He, et al. 2024. Actions speak louder than words: Trillion-parameter sequential transducers for generative recommendations. *arXiv preprint arXiv:2402.17152* (2024).

[34] Hanrong Zhang, Shicheng Fan, Henry Peng Zou, Yankai Chen, Zhenting Wang, Jiayu Zhou, Chengze Li, Wei-Chieh Huang, Yifei Yao, Kening Zheng, et al. 2026. Coevoskills: Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687* (2026).

[35] Song Zhang, Nan Zheng, and Danli Wang. 2022. GBERT: Pre-training user representations for ephemeral group recommendation. In *Proceedings of the 31st ACM international conference on information & knowledge management*. 2631–2639.

[36] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM international conference on information & knowledge management*. 1893–1902.